

Адаптация под разные размеры. Динамической навигация

Введение

Одно из главных преимуществ разработки приложения на платформе Android - это широкие возможности для пользователей различных форм-факторов, таких как носимые и складные устройства, планшеты, настольные компьютеры и даже телевизоры. При использовании приложения ваши пользователи могут захотеть использовать это приложение на устройствах с большим экраном, чтобы воспользоваться преимуществами увеличенной площади. Пользователи Android все чаще используют свои приложения на нескольких устройствах с разным размером экрана и ожидают высококачественного пользовательского опыта на всех устройствах.

До сих пор вы учились создавать приложения в основном для мобильных устройств. В этом уроке вы узнаете, как преобразовать свои приложения, чтобы сделать их адаптивными к экранам других размеров. Вы будете использовать шаблоны адаптивной навигации, которые красивы и удобны как для мобильных устройств, так и для устройств с большим экраном, таких как раскладушки, планшеты и настольные компьютеры.

Необходимые условия

- Знакомство с программированием на Kotlin, включая классы, функции и условия
- Опыт использования классов ViewModel
- Опыт создания композитных функций
- Опыт создания макетов с помощью Jetpack Compose
- Опыт запуска приложений на устройстве или эмуляторе

Что вы узнаете

- Как создать навигацию между экранами без Navigation Graph для простых приложений
- Как создать адаптивный макет навигации с помощью Jetpack Compose
- Как создать пользовательский обработчик возврата

Что вы создадите

- Вы внедрите динамическую навигацию в существующее приложение **Reply**, чтобы его макет адаптировался под все размеры экранов.
- Готовый продукт будет выглядеть так, как показано на изображении ниже:

Что вам понадобится Компьютер с доступом в интернет, веб-браузер и Android Studio Доступ к GitHub

Обзор приложений. Введение в приложение Reply

Приложение Reply - это мультиэкранное приложение, напоминающее почтовый клиент.

Оно содержит 4 различные категории, которые отображаются на разных вкладках, а именно: входящие, отправленные, черновики и спам.

Загрузите начальный код В Android Studio откройте папку basic-android-kotlin-compose-training-reply-app.

URL стартового кода: <https://github.com/google-developer-training/basic-android-kotlin-compose-training-reply-app> Название ветки со стартовым кодом: starter

Обзор стартового кода

Важные директории в приложении Reply

Слой данных и слой пользовательского интерфейса в проекте Reply app разделены по разным директориям. ReplyViewModel, ReplyUiState и другие составные элементы находятся в директории ui. Классы data и enum, определяющие слой данных, а также классы поставщиков данных находятся в каталоге data.

Инициализация данных в приложении Reply

Приложение Reply инициализируется данными с помощью метода initializeUIState() в модели ReplyViewModel, который выполняется в функции init.

ReplyViewModel.kt

```
...
    init {
        initializeUIState()
    }

    private fun initializeUIState() {
        var mailboxes: Map<MailboxType, List<Email>> =
            LocalEmailsDataProvider.allEmails.groupBy { it.mailbox }
        _uiState.value = ReplyUiState(
            mailboxes = mailboxes,
            currentSelectedEmail = mailboxes[MailboxType.Inbox]?.get(0)
                ?: LocalEmailsDataProvider.defaultEmail
        )
    }
...

```

Композит уровня экрана Как и в других приложениях, в приложении Reply в качестве основного composable используется ReplyApp composable, в котором объявляются viewModel и uiState. Различные функции viewModel() также передаются в качестве лямбда-аргументов для композита ReplyHomeScreen.

ReplyApp.kt

```

...
@Composable
fun ReplyApp(modifier: Modifier = Modifier) {
    val viewModel: ReplyViewModel = viewModel()
    val replyUiState = viewModel.uiState.collectAsState().value

    ReplyHomeScreen(
        replyUiState = replyUiState,
        onTabPressed = { mailboxType: MailboxType ->
            viewModel.updateCurrentMailbox(mailboxType = mailboxType)
            viewModel.resetHomeScreenStates()
        },
        onEmailCardPressed = { email: Email ->
            viewModel.updateDetailsScreenStates(
                email = email
            )
        },
        onDetailScreenBackPressed = {
            viewModel.resetHomeScreenStates()
        },
        modifier = modifier
    )
}

```

Другие составные элементы

- ReplyHomeScreen.kt: содержит составные элементы экрана для главного экрана, включая элементы навигации. ReplyHomeContent.kt: содержит составные элементы, определяющие более подробные составные элементы главного экрана.
- ReplyDetailsScreen.kt: содержит составные элементы экрана и меньшие составные элементы для экрана подробностей.

Не стесняйтесь подробно изучить каждый файл, чтобы лучше понять составные элементы, прежде чем приступать к следующему разделу урока.

Смена экранов без навигационного графа

В предыдущем разделе вы научились использовать класс `NavHostController` для перехода с одного экрана на другой. С помощью Compose вы также можете менять экраны с помощью простых условных операторов, используя изменяемые во время выполнения состояния. Это особенно полезно в небольших приложениях, таких как приложение Reply, где требуется переключаться только между двумя экранами.

Смена экранов с помощью изменения состояния В Compose экраны перекомпонуются при изменении состояния. Вы можете менять экраны, используя простые условия, чтобы реагировать на изменения состояний.

Вы будете использовать условия для отображения содержимого главного экрана, когда пользователь находится на главном экране, и экрана подробностей, когда пользователь не находится на главном экране.

Модифицируйте приложение Reply, чтобы оно позволяло менять экран при изменении состояния, выполнив следующие действия:

Откройте начальный код в Android Studio. В составной части ReplyHomeScreen в файле ReplyHomeScreen.kt оберните составную часть ReplyAppContent оператором if для случая, когда свойство isShowingHomepage объекта replyUiState равно true.

ReplyHomeScreen.kt

```
@Composable
fun ReplyHomeScreen(
    replyUiState: ReplyUiState,
    onTabPressed: (MailboxType) -> Unit,
    onEmailCardPressed: (Int) -> Unit,
    onDetailScreenBackPressed: () -> Unit,
    modifier: Modifier = Modifier
) {
    ...
    if (replyUiState.isShowingHomepage) {
        ReplyAppContent(
            replyUiState = replyUiState,
            onTabPressed = onTabPressed,
            onEmailCardPressed = onEmailCardPressed,
            navigationItemContentList = navigationItemContentList,
            modifier = modifier
        )
    }
}
```

Теперь вы должны учесть сценарий, когда пользователь не находится на главном экране, показывая экран с подробной информацией.

Добавьте ветку else с композитом ReplyDetailsScreen в ее теле. Добавьте replyUiState, onDetailScreenBackPressed и модификатор в качестве аргументов для составного ReplyDetailsScreen. ReplyHomeScreen.kt

```
@Composable
fun ReplyHomeScreen(
    replyUiState: ReplyUiState,
    onTabPressed: (MailboxType) -> Unit,
    onEmailCardPressed: (Int) -> Unit,
    onDetailScreenBackPressed: () -> Unit,
    modifier: Modifier = Modifier
) {
    ...

    if (replyUiState.isShowingHomepage) {
```

```

        ReplyAppContent(
            replyUiState = replyUiState,
            onTabPressed = onTabPressed,
            onEmailCardPressed = onEmailCardPressed,
            navigationItemContentList = navigationItemContentList,
            modifier = modifier
        )
    } else {
        ReplyDetailsScreen(
            replyUiState = replyUiState,
            onBackPressed = onDetailScreenBackPressed,
            modifier = modifier
        )
    }
}

```

Объект `replyUiState` является объектом состояния. Поэтому при изменении свойства `isShowingHomepage` объекта `replyUiState` композит `ReplyHomeScreen` перекомпилируется, а оператор `if/else` переоценивается во время выполнения. Такой подход поддерживает навигацию между различными экранами без использования класса `NavHostController`.

Создайте пользовательский обработчик возврата

Одно из преимуществ использования композитного `NavHost` для переключения между экранами заключается в том, что направления предыдущих экранов сохраняются в бэжстэке. Эти сохраненные экраны позволяют системной кнопке «Назад» легко переходить к предыдущему экрану при вызове. Поскольку приложение `Reply` не использует `NavHost`, вам придется добавить код для обработки кнопки «Назад» вручную. Вы сделаете это следующим образом.

Выполните следующие шаги, чтобы создать пользовательский обработчик кнопки «Назад» в приложении «Ответить»:

В первой строке составного экрана `ReplyDetailsScreen` добавьте составной обработчик `BackHandler`. Вызовите функцию `onBackPressed()` в теле композита `BackHandler`. `ReplyDetailsScreen.kt`

```

...
import androidx.activity.compose.BackHandler
...
@Composable
fun ReplyDetailsScreen(
    replyUiState: ReplyUiState,
    onBackPressed: () -> Unit,
    modifier: Modifier = Modifier
) {
    BackHandler {
        onBackPressed()
    }
}

```

```
}  
...
```

5. Запустите приложение на устройствах с большим экраном

Проверьте свое приложение с помощью эмулятора с изменяемым размером экрана. Чтобы создавать удобные приложения, разработчики должны понимать, как пользователи воспринимают различные форм-факторы. Поэтому с самого начала процесса разработки необходимо тестировать приложения на различных форм-факторах.

Для достижения этой цели можно использовать множество эмуляторов с разными размерами экрана. Однако это может быть обременительно, особенно если вы создаете приложение сразу для нескольких размеров экрана. Вам также может понадобиться проверить, как работающее приложение реагирует на изменение размера экрана, например, изменение ориентации, изменение размера окна на рабочем столе или изменение состояния складки в foldable.

Android Studio поможет вам протестировать эти сценарии с помощью эмулятора с изменяемым размером экрана.

Для настройки эмулятора с изменяемым размером выполните следующие действия:

В Android Studio выберите Инструменты > Диспетчер устройств.

В диспетчере устройств нажмите значок +, чтобы создать виртуальное устройство.

Выберите категорию Phone и устройство Resizable (Experimental). Нажмите кнопку Далее.

Выберите уровень API 34 или выше. Нажмите Далее.

Назовите новое виртуальное устройство Android. Нажмите Готово.

Запуск приложения на эмуляторе с большим экраном. Теперь, когда вы настроили эмулятор с изменяемым размером, давайте посмотрим, как приложение выглядит на большом экране.

Запустите приложение на эмуляторе с изменяемым размером экрана. В качестве режима отображения выберите Tablet.

![] (Запуск приложения на эмуляторе с большим экраном. Теперь, когда вы настроили эмулятор с возможностью изменения размеров, давайте посмотрим, как приложение выглядит на большом

экране.

Запустите приложение на эмуляторе с изменяемым размером экрана. Выберите режим отображения «Планшет»).

Осмотрите приложение в режиме планшета в ландшафтном режиме.

![] (Обратите внимание, что экран планшета вытянут по горизонтали. Хотя такая ориентация функционально работает, возможно, это не лучшее использование большой площади экрана. Давайте разберемся с этим дальше.

Дизайн для больших экранов При взгляде на это приложение на планшете первой вашей мыслью может быть то, что оно плохо оформлено и непривлекательно. Вы совершенно правы: этот макет не предназначен для использования на больших экранах.

При разработке дизайна для больших экранов, таких как планшеты и раскладушки, необходимо учитывать эргономику пользователя и близость его пальцев к экрану. На мобильных устройствах пальцы пользователя могут легко дотянуться до большей части экрана, поэтому расположение интерактивных элементов, таких как кнопки и элементы навигации, не так критично. Однако на больших экранах расположение важных интерактивных элементов в центре экрана может затруднить их доступность.

Как вы видите на примере приложения Reply, дизайн для больших экранов - это не просто растягивание или увеличение элементов пользовательского интерфейса, чтобы они поместились на экране. Это возможность использовать увеличенную площадь для создания другого опыта для ваших пользователей. Например, вы можете добавить еще один макет на тот же экран, чтобы предотвратить необходимость перехода на другой экран или сделать возможной многозадачность).

Такой дизайн может повысить продуктивность работы пользователей и способствовать их вовлеченности. Но прежде чем внедрять этот дизайн, необходимо научиться создавать различные макеты для экранов разных размеров.

Примечание: Портретная ориентация является основной для телефонов, а альбомная - для планшетов. При разработке адаптивного дизайна на соответствующий размер экрана влияют текущий размер окна и ориентация устройства, а не только его размер.

6. Сделайте макет адаптивным к разным размерам экрана

Что такое точки останова? Вам может быть интересно, как можно показать разные макеты для одного и того же приложения. Короткий ответ - с помощью условных сигналов для разных состояний, как вы делали в начале этого коделаба.

Чтобы создать адаптивное приложение, вам нужно, чтобы макет менялся в зависимости от размера экрана. Точка измерения, в которой меняется макет, называется точкой останова. В Material Design был

создан определенный диапазон точек останова, который охватывает большинство экранов Android.

Эта таблица показывает, что, например, если ваше приложение работает на устройстве с размером экрана менее 600 dp, вы должны показывать мобильный макет.

Примечание: понятие точек останова в адаптивных макетах отличается от понятия точек останова в отладке.

Использование классов размеров окон API `WindowSizeClass`, представленный в Compose, упрощает реализацию точек останова Material Design.

Window Size Classes вводит три категории размеров: Compact, Medium и Expanded, как для ширины, так и для высоты.

Выполните следующие шаги, чтобы реализовать API `WindowSizeClass` в приложении Reply:

Добавьте зависимость `material3-window-size-class` в файл `build.gradle.kts` модуля. `build.gradle.kts`

```
...
dependencies {
    ...
    implementation("androidx.compose.material3:material3-window-size-class")
    ...
}
```

Нажмите Sync Now, чтобы синхронизировать gradle после добавления зависимости.

Теперь, когда файл `build.gradle.kts` обновлен, вы можете создать переменную, которая будет хранить размер окна приложения в любой момент времени.

В функции `onCreate()` в файле `MainActivity.kt` назначьте метод `calculateWindowSizeClass()` с данным контекстом, переданным в качестве параметра, переменной с именем `windowSize`. Импортируйте соответствующий пакет `calculateWindowSizeClass`. `MainActivity.kt`

```
...
import androidx.compose.material3.windowSizeClass.calculateWindowSizeClass
...

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    setContent {
        ReplyTheme {
            val layoutDirection = LocalLayoutDirection.current
        }
    }
}
```



```

        Surface (
            // ...
        ) {
            val windowSize = calculateWindowSizeClass(this)
            ReplyApp()
        }
    }
    ...

```

Обратите внимание на красное подчеркивание синтаксиса `calculateWindowSizeClass`, которое показывает красную лампочку. Щелкните на красной лампочке слева от переменной `windowSize` и выберите `Opt in for 'ExperimentalMaterial3WindowSizeClassApi' on 'onCreate'`, чтобы создать аннотацию поверх метода `onCreate()`.

Примечание: Это сообщение об ошибке появляется потому, что в настоящее время API `material3-window-size-class` все еще находится в альфа-версии, которая требует этой аннотации.

Вы можете использовать переменную `WindowWidthSizeClass` в `MainActivity.kt`, чтобы определить, какой макет отображать в различных композитах. Давайте подготовим композит `ReplyApp` для получения этого значения.

В файле `ReplyApp.kt` измените компонент `ReplyApp`, чтобы он принимал `WindowWidthSizeClass` в качестве параметра, и импортируйте соответствующий пакет.

`ReplyApp.kt`

```

...
import androidx.compose.material3.windowSizeClass.WindowWidthSizeClass
...
@Composable
fun ReplyApp(
    windowSize: WindowWidthSizeClass,
    modifier: Modifier = Modifier
) {
    ...
}

```

Передайте переменную `windowSize` компоненту `ReplyApp` в методе `onCreate()` файла `MainActivity.kt`.

`MainActivity.kt`

```

...
setContent {
    ReplyTheme {
        Surface {
            val windowSize = calculateWindowSizeClass(this)
            ReplyApp(
                windowSize = windowSize.widthSizeClass
            )
        }
    }
}
...

```

Вам также необходимо обновить предварительный просмотр приложения для параметра `windowSize`.

Передайте `WindowWidthSizeClass.Compact` в качестве параметра `windowSize` компоненту `ReplyApp` `composable` для компонента предварительного просмотра и импортируйте соответствующий пакет.

MainActivity.kt

```
...
import androidx.compose.material3.windowSizeClass.WindowWidthSizeClass
...

@Preview(showBackground = true)
@Composable
fun ReplyAppCompactPreview() {
    ReplyTheme {
        Surface {
            ReplyApp(
                windowSize = WindowWidthSizeClass.Compact,
            )
        }
    }
}
```

Чтобы изменить макеты приложения в зависимости от размера экрана, добавьте в `ReplyApp` оператор `when`, основанный на значении `WindowWidthSizeClass`. `ReplyApp.kt`

```
...

@Composable
fun ReplyApp(
    windowSize: WindowWidthSizeClass,
    modifier: Modifier = Modifier
) {
    val viewModel: ReplyViewModel = viewModel()
    val replyUiState = viewModel.uiState.collectAsState().value

    when (windowSize) {
        WindowWidthSizeClass.Compact -> {
        }
        WindowWidthSizeClass.Medium -> {
        }
        WindowWidthSizeClass.Expanded -> {
        }
        else -> {
        }
    }
}

...
```

На этом этапе вы заложили основу для использования значений `WindowSizeClass` для изменения макетов в вашем приложении. Следующий шаг - определить, как ваше приложение должно выглядеть на экранах разных размеров.

7. Реализация адаптивного макета навигации

Реализация адаптивной навигации пользовательского интерфейса В настоящее время для всех размеров экрана используется нижняя навигация.

Как уже говорилось ранее, этот элемент навигации не является идеальным, поскольку пользователям может быть трудно добраться до этих важных элементов навигации на больших экранах. К счастью, существуют рекомендуемые шаблоны для различных элементов навигации для разных классов размеров окон в навигации для отзывчивых пользовательских интерфейсов. Для приложения Reply можно использовать следующие элементы:

Навигационный рельс - еще один навигационный компонент Material Design, который позволяет сделать компактные навигационные опции для основных направлений доступными со стороны приложения.

Аналогично, постоянный/непостоянный навигационный ящик создается с помощью материального дизайна как еще один вариант обеспечения эргономичного доступа для больших экранов.

Реализация навигационного ящика Чтобы создать навигационный ящик для расширенных экранов, можно использовать параметр `navigationType`. Для этого выполните следующие действия:

Для представления различных типов элементов навигации создайте новый файл `WindowStateUtils.kt` в новом пакете `utils`, который находится в каталоге `ui`. Добавьте класс Enum для представления различных типов элементов навигации. `WindowStateUtils.kt`

```
package com.example.reply.ui.utils

enum class ReplyNavigationType {
    BOTTOM_NAVIGATION, NAVIGATION_RAIL, PERMANENT_NAVIGATION_DRAWER
}
```

Чтобы успешно реализовать навигационный ящик, необходимо определить тип навигации в зависимости от размера окна приложения.

В составном `ReplyApp` создайте переменную `navigationType` и присвойте ей соответствующее значение `ReplyNavigationType` в соответствии с размером экрана в операторе `when`. `ReplyApp.kt`

```

...
import com.example.reply.ui.utils.ReplyNavigationType
...
val navigationType: ReplyNavigationType
when (windowSize) {
    WindowWidthSizeClass.Compact -> {
        navigationType = ReplyNavigationType.BOTTOM_NAVIGATION
    }
    WindowWidthSizeClass.Medium -> {
        navigationType = ReplyNavigationType.NAVIGATION_RAIL
    }
    WindowWidthSizeClass.Expanded -> {
        navigationType = ReplyNavigationType.PERMANENT_NAVIGATION_DRAWER
    }
    else -> {
        navigationType = ReplyNavigationType.BOTTOM_NAVIGATION
    }
}
...

```

Вы можете использовать значение `navigationType` в композите `ReplyHomeScreen`. Вы можете подготовиться к этому, сделав его параметром для композита.

В составном `ReplyHomeScreen` добавьте `navigationType` в качестве параметра. `ReplyHomeScreen.kt`

```

...
@Composable
fun ReplyHomeScreen(
    navigationType: ReplyNavigationType,
    replyUiState: ReplyUiState,
    onTabPressed: (MailboxType) -> Unit,
    onEmailCardPressed: (Email) -> Unit,
    onDetailScreenBackPressed: () -> Unit,
    modifier: Modifier = Modifier
)
...

```

Передайте `navigationType` в составной `ReplyHomeScreen`. `ReplyApp.kt`

```

...
ReplyHomeScreen(
    navigationType = navigationType,
    replyUiState = replyUiState,
    onTabPressed = { mailboxType: MailboxType ->
        viewModel.updateCurrentMailbox(mailboxType = mailboxType)
        viewModel.resetHomeScreenStates()
    }
)

```

```

    },
    onEmailCardPressed = { email: Email ->
        viewModel.updateDetailsScreenStates(
            email = email
        )
    },
    onDetailScreenBackPressed = {
        viewModel.resetHomeScreenStates()
    },
    modifier = modifier
)
...

```

Далее вы можете создать ветку для отображения содержимого приложения с навигационным ящиком, когда пользователь открывает приложение на расширенном экране и отображает домашний экран.

В составном теле `ReplyHomeScreen` добавьте оператор `if` для условия `navigationType == ReplyNavigationType.PERMANENT_NAVIGATION_DRAWER && replyUiState.isShowingHomepage`. `ReplyHomeScreen.kt`

```

import androidx.compose.material3.PermanentNavigationDrawer
...
@Composable
fun ReplyHomeScreen(
    navigationType: ReplyNavigationType,
    replyUiState: ReplyUiState,
    onTabPressed: (MailboxType) -> Unit,
    onEmailCardPressed: (Email) -> Unit,
    onDetailScreenBackPressed: () -> Unit,
    modifier: Modifier = Modifier
) {
    ...
    if (navigationType == ReplyNavigationType.PERMANENT_NAVIGATION_DRAWER
        && replyUiState.isShowingHomepage
    ) {
    }

    if (replyUiState.isShowingHomepage) {
        ReplyAppContent(
            replyUiState = replyUiState,
            ...

```

Чтобы создать постоянный ящик, создайте композит `PermanentNavigationDrawer` в теле оператора `if` и добавьте композит `NavigationDrawerContent` в качестве входного параметра `drawerContent`. Добавьте составной элемент `ReplyAppContent` в качестве заключительного лямбда-аргумента `PermanentNavigationDrawer`. `ReplyHomeScreen.kt`

```

...
    if (navigationType == ReplyNavigationType.PERMANENT_NAVIGATION_DRAWER
        && replyUiState.isShowingHomepage
    ) {
        PermanentNavigationDrawer(
            drawerContent = {

PermanentDrawerSheet(Modifier.width(dimensionResource(R.dimen.drawer_width))) {
                NavigationDrawerContent(
                    selectedDestination = replyUiState.currentMailbox,
                    onTabPressed = onTabPressed,
                    navigationItemContentList = navigationItemContentList,
                    modifier = Modifier
                        .wrapContentWidth()
                        .fillMaxHeight()

                .background(MaterialTheme.colorScheme.inverseOnSurface)

                .padding(dimensionResource(R.dimen.drawer_padding_content))
                )
            }
        ) {
            ReplyAppContent(
                replyUiState = replyUiState,
                onTabPressed = onTabPressed,
                onEmailCardPressed = onEmailCardPressed,
                navigationItemContentList = navigationItemContentList,
                modifier = modifier
            )
        }
    }
}
...

```

Добавьте ветку else, которая использует предыдущее составное тело, чтобы сохранить предыдущее разветвление для нерасширенных экранов. ReplyHomeScreen.kt

```

...
    if (navigationType == ReplyNavigationType.PERMANENT_NAVIGATION_DRAWER
        && replyUiState.isShowingHomepage
    ) {
        PermanentNavigationDrawer(
            drawerContent = {

PermanentDrawerSheet(Modifier.width(dimensionResource(R.dimen.drawer_width))) {
                NavigationDrawerContent(
                    selectedDestination = replyUiState.currentMailbox,
                    onTabPressed = onTabPressed,
                    navigationItemContentList = navigationItemContentList,
                    modifier = Modifier

```

```

                .wrapContentWidth()
                .fillMaxHeight()

            .background(MaterialTheme.colorScheme.inverseOnSurface)

            .padding(dimensionResource(R.dimen.drawer_padding_content))
        )
    }
}
) {
    ReplyAppContent(
        replyUiState = replyUiState,
        onTabPressed = onTabPressed,
        onEmailCardPressed = onEmailCardPressed,
        navigationItemContentList = navigationItemContentList,
        modifier = modifier
    )
}
} else {
    if (replyUiState.isShowingHomepage) {
        ReplyAppContent(
            replyUiState = replyUiState,
            onTabPressed = onTabPressed,
            onEmailCardPressed = onEmailCardPressed,
            navigationItemContentList = navigationItemContentList,
            modifier = modifier
        )
    } else {
        ReplyDetailsScreen(
            replyUiState = replyUiState,
            onBackPressed = onDetailScreenBackPressed,
            modifier = modifier
        )
    }
}
}
...

```

Запустите приложение в режиме планшета. Вы должны увидеть следующий экран:

Реализация навигационного рельса Аналогично реализации навигационного ящика, вам нужно использовать параметр `navigationType` для переключения между элементами навигации.

Сначала добавим навигационный рельс для средних экранов.

Начните с подготовки композита `ReplyAppContent`, добавив `navigationType` в качестве параметра. `ReplyHomeScreen.kt`

```

...
@Composable

```

```
private fun ReplyAppContent(
    navigationType: ReplyNavigationType,
    replyUiState: ReplyUiState,
    onTabPressed: ((MailboxType) -> Unit),
    onEmailCardPressed: (Email) -> Unit,
    navigationItemContentList: List<NavigationItemContent>,
    modifier: Modifier = Modifier
) {
    ...
}
```

Передайте значение `navigationType` в оба компонента `ReplyAppContent`. `ReplyHomeScreen.kt`

```
...
        ReplyAppContent(
            navigationType = navigationType,
            replyUiState = replyUiState,
            onTabPressed = onTabPressed,
            onEmailCardPressed = onEmailCardPressed,
            navigationItemContentList = navigationItemContentList,
            modifier = modifier
        )
    }
} else {
    if (replyUiState.isShowingHomepage) {
        ReplyAppContent(
            navigationType = navigationType,
            replyUiState = replyUiState,
            onTabPressed = onTabPressed,
            onEmailCardPressed = onEmailCardPressed,
            navigationItemContentList = navigationItemContentList,
            modifier = modifier
        )
    }
}
...
```

Далее добавим ветвление, которое позволяет приложению отображать навигационные рельсы для некоторых сценариев.

В первой строке композитного тела `ReplyAppContent` оберните композит `ReplyNavigationRail` вокруг композита `AnimatedVisibility` и установите параметр `visible` равным `true`, если значение `ReplyNavigationType` равно `NAVIGATION_RAIL`. `ReplyHomeScreen.kt`

```
...
@Composable
private fun ReplyAppContent(
    navigationType: ReplyNavigationType,
    replyUiState: ReplyUiState,
    onTabPressed: ((MailboxType) -> Unit),
    onEmailCardPressed: (Email) -> Unit,
    navigationItemContentList: List<NavigationItemContent>,
    modifier: Modifier = Modifier
) {
    ...
}
```



```

        modifier: Modifier = Modifier
    ) {
        Box(modifier = modifier) {
            AnimatedVisibility(visible = navigationType ==
ReplyNavigationType.NAVIGATION_RAIL) {
                ReplyNavigationRail(
                    currentTab = replyUiState.currentMailbox,
                    onTabPressed = onTabPressed,
navigationItemContentList = navigationItemContentList
                )
            }
            Column(
                modifier = Modifier
                    .fillMaxSize()
                    .background(
                        MaterialTheme.colorScheme.inverseOnSurface
                    )
            ) {
                ReplyListOnlyContent(
                    replyUiState = replyUiState,
                    onEmailCardPressed = onEmailCardPressed,
                    modifier = Modifier.weight(1f)
                        .padding(
                            horizontal =
dimensionResource(R.dimen.email_list_only_horizontal_padding)
                        )
                )
                ReplyBottomNavigationBar(
                    currentTab = replyUiState.currentMailbox,
                    onTabPressed = onTabPressed,
                    navigationItemContentList = navigationItemContentList,
                    modifier = Modifier
                        .fillMaxWidth()
                )
            }
        }
    }
    ...

```

Чтобы правильно выровнять композиты, оберните композит `AnimatedVisibility` и композит `Column`, находящиеся в теле `ReplyAppContent`, в композит `Row`. `ReplyHomeScreen.kt`

```

...
@Composable
private fun ReplyAppContent(
    navigationType: ReplyNavigationType,
    replyUiState: ReplyUiState,
    onTabPressed: ((MailboxType) -> Unit),
    onEmailCardPressed: (Email) -> Unit,
    navigationItemContentList: List<NavigationItemContent>,
    modifier: Modifier = Modifier,

```

```

    ) {
        Row(modifier = modifier) {
            AnimatedVisibility(visible = navigationType ==
ReplyNavigationType.NAVIGATION_RAIL) {
                val navigationRailContentDescription =
stringResource(R.string.navigation_rail)
                ReplyNavigationRail(
                    currentTab = replyUiState.currentMailbox,
                    onTabPressed = onTabPressed,
                    navigationItemContentList = navigationItemContentList
                )
            }
            Column(
                modifier = Modifier
                    .fillMaxSize()
                    .background(MaterialTheme.colorScheme.inverseOnSurface)
            ) {
                ReplyListOnlyContent(
                    replyUiState = replyUiState,
                    onEmailCardPressed = onEmailCardPressed,
                    modifier = Modifier.weight(1f)
                        .padding(
                            horizontal =
dimensionResource(R.dimen.email_list_only_horizontal_padding)
                        )
                )
                ReplyBottomNavigationBar(
                    currentTab = replyUiState.currentMailbox,
                    onTabPressed = onTabPressed,
                    navigationItemContentList = navigationItemContentList,
                    modifier = Modifier
                        .fillMaxWidth()
                )
            }
        }
    }
    ...

```

Наконец, давайте убедимся, что нижняя навигация отображается в некоторых сценариях.

После составного элемента ReplyListOnlyContent оберните составной элемент ReplyBottomNavigationBar составным элементом AnimatedVisibility. Установите параметр visible, если значение ReplyNavigationType равно BOTTOM_NAVIGATION. ReplyHomeScreen.kt

```

...
ReplyListOnlyContent(
    replyUiState = replyUiState,
    onEmailCardPressed = onEmailCardPressed,
    modifier = Modifier.weight(1f)
        .padding(

```

```
        horizontal =  
        dimensionResource(R.dimen.email_list_only_horizontal_padding)  
    )  
  
    )  
    AnimatedVisibility(visible = navigationType ==  
        ReplyNavigationType.BOTTOM_NAVIGATION) {  
        val bottomNavigationContentDescription =  
            stringResource(R.string.navigation_bottom)  
        ReplyBottomNavigationBar(  
            currentTab = replyUiState.currentMailbox,  
            onTabPressed = onTabPressed,  
            navigationItemContentList = navigationItemContentList,  
            modifier = Modifier  
                .fillMaxWidth()  
        )  
    }  
  
    ...
```

Запустите приложение в режиме раскладывания Unfolded. Вы должны увидеть следующий экран:

8. Получение кода решения

Чтобы загрузить код готового коделаба, вы можете воспользоваться этими командами git:

```
git clone https://github.com/google-developer-training/basic-android-kotlin-  
compose-training-reply-app.git  
cd basic-android-kotlin-compose-training-reply-app  
git checkout nav-update
```

Также вы можете скачать репозиторий в виде zip-файла, распаковать его и открыть в Android Studio.

9. Заключение

Поздравляем! Вы стали на шаг ближе к тому, чтобы сделать приложение **Reply** адаптивным для всех размеров экрана, реализовав адаптивный макет навигации. Вы улучшили пользовательский опыт, используя многие форм-факторы Android. В следующем коделабе вы продолжите совершенствовать свои навыки работы с адаптивными приложениями, внедряя адаптивное расположение контента, тестирование и предварительные просмотры.